CSED233 · 데이터구조 · 포스텍

Ch3. Algorithm Analysis — Big-O & 점근 분석

1. Algorithm(알고리즘) 비교: 왜 필요한가

이론 & 배경

- **Algorithm**: 유한 시간 내 문제를 해결하는 step-by-step 절차.
- 같은 문제에 여러 알고리즘 → 어느 것이 더 나은가?
- **비교 척도 = 효율성(Efficiency)**
  - **Time complexity(시간복잡도)**: 실행 시간
  - **Space complexity(공간복잡도)**: 메모리 사용량

두 가지 비교 방법

방법	장점	한계
Empirical(경험적 분석) (실제 실행)	구체적 성능 수치	구현 비용, 입력 의존, 동일 HW/SW 필요
Theoretical(이론적 분석) (이론 분석)	HW/SW 독립, 모든 입력 고려	상수항 무시로 실제와 괴리 가능

- 이론 분석: 입력 크기 n에 대한 running time/space를 **함수**로 표현.

2. 점근 분석 (Asymptotic Analysis)

원리

"극한 행동(limiting behavior)만 본다" — $n \to \infty$ 관점.

1. **Lower-order term 무시**

- 예: $f(n) = n^2 + 3n \to n$ 커지면 $3n$은 n^2에 비해 미미

- → $f(n) \sim n^2$ (asymptotically equivalent)

2. **상수 무시**

- 예: $6N^2$ vs $20N^2$ → $N \gg 20$ 이면 N^2 자체가 증가 주체
- 상수 6, 20은 중요치 않음

Best(최선) / Worst(최악) / Average(평균)

Case	의미	분석 용이성
Best(최선)	크기 n인 입력 중 실행 비용의 최솟값	참고용
Average(평균)	명시한 입력 확률분포에서 실행 비용의 기댓값	결정 어려움
Worst(최악)	크기 n인 입력 중 실행 비용의 최댓값	**쉬움, 실시간 앱 critical**

- 이론 분석은 **대부분 worst case** 집중.
- Best/Avg/Worst는 어떤 입력 사례의 비용을 분석하는지 구분하고, O/Ω/Θ는 분석한 비용 함수의 점근적 상한·하한·타이트 경계를 표현한다. 세 경우 모두 O, Ω, Θ를 사용할 수 있으며, Average-case 분석에는 입력의 확률분포를 명시해야 한다.

3. Big-O(빅오) 정의 & 성질

정의

> $T(n) = O(f(n)) \iff \exists\ c > 0, \exists\ n_0 \geq 1$ s.t. $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$.

- **Upper bound**(증가율 상한)
- Big-Ω: $T(n) \geq c \cdot f(n)$ (∃ c>0, ∃ $n_0 \geq 1$, 모든 $n \geq n_0$) → **lower bound**
- Big-Θ: T(n)∈Θ(f(n))은 충분히 큰 모든 n에서 $c_1 f(n) \leq T(n) \leq c_2 f(n)$을 만족하는 양의 상수 c_1, c_2가 존재한다는 뜻이다. 즉 두 함수가 같은 점근적 차수를 가진다. 반면 T(n)~f(n)은 f(n)>0인 충분히 큰 n에 대해 lim(n→∞) T(n)/f(n)=1을 의미하는 더 강한 조건이다.

대표 예시 — 유도 과정 포함

예 1: $T(n) = (n+1)^2$ 이 $O(n^2)$ 인가?

```
(n+1)² = n² + 2n + 1
    ≤ n² + 2n² + n²   (∵ n ≥ 1 이면 1 ≤ n², 2n ≤ 2n²)
    = 4n²
→ c = 4, n₀ = 1 로 성립 ⟹ O(n²) ✓
```

기타:

- $3n^3 \in O(n^3)$ (tight) ↔ $3n^3 \in O(n^4)$ (loose)
- $3n^3 + 2n^2 + 8 \in O(n^3)$
- $2^{100} \in O(1)$ (상수)
- $3\log n + 5 \in O(\log n)$

4. 성장률 순위 & Properties

Typical Growth Rate (작은 순)

대표 함수의 점근적 성장률은 n→∞에서 다음 순서로 증가한다. 이때 로그의 밑은 1보다 큰 고정 상수이다. $1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$ 여기서 f≺g는 f(n)=o(g(n)), 즉 $\lim(n\to\infty)$ f(n)/g(n)=0을 의미한다. Big-O 집합으로 표현하면 각 항 사이에는 진부분집합 관계 ⊊를 사용한다.

Big-O(빅오) 연산 규칙

- **Addition**: $T_1 \in O(f), T_2 \in O(g) \Rightarrow T_1 + T_2 \in O(\max f, g)$
- **Product**: $T_1 \in O(f), T_2 \in O(g) \Rightarrow T_1 \cdot T_2 \in O(f \cdot g)$
- 상수배 법칙: T(n)∈O(f(n))이고 a>0이 n과 무관한 고정 상수이면 aT(n)∈O(f(n))이다. 따라서 고정 상수배는 점근적 시간복잡도의 차수를 바꾸지 않는다.
- 추이성: $T \in O(f), f \in O(g) \Rightarrow T \in O(g)$
- 다항식 차수 $d \to T(n) \in O(n^d)$

5. 실제 분석 방법

기본 가정

Basic operations = constant time:

- 산술 연산 (fixed-width)
- Assignment
- Array(배열) index 접근
- 단순 값 비교

조합 규칙:

- 연속 실행되는 문장의 비용은 합산한다.
- 루프의 비용은 각 반복에서 발생하는 본문 비용을 모두 합산하고 반복 제어 비용을 더하여 구한다. 특히 모든 반복의 본문 비용이 동일한 점근적 차수를 가지는 경우에는 (반복 횟수) × (반복당 비용)으로 계산할 수 있다. 반복마다 비용이 달라지면 합 Σ_i B(i)를 직접 계산한다.

예시: 3중 행렬 곱셈

```
for i := 1 to n
 for j := 1 to n begin
   C[i,j] := 0
   for k := 1 to n
     C[i,j] := C[i,j] + A[i,k]*B[k,j]
 end
```

유도:

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} (c_1 + \sum_{k=1}^{n} c_2) = c_1 n^2 + c_2 n^3$$

→ $T(n) \in O(n^3)$

---

6. 점근 분석의 한계

주의점

- **상수 factor 무시** → 실제로는 상수가 지배할 수 있음
  - 예: $1000n$ vs n^2 → $n < 1000$에선 n^2이 더 빠름
- **메모리 계층(cache/MM/HDD) 무시**
- 실제 실행 시간은 산술 연산 횟수뿐 아니라 캐시 적중률, 메모리 접근 패턴, 메모리 지연시간 등의 영향을 받는다. 특히 캐시 미스가 발생하여 메인 메모리에 접근하는 경우에는 단순 산술 연산보다 큰 비용이 들 수 있다. 따라서 점근적 연산 수가 적은 알고리즘이 실제 실행에서도 항상 빠른 것은 아니다.

Q1. 루프 안의 if가 일부 iteration에서만 실행될 때 점근 분석은 어떻게 하나?

> 힌트

점근 분석의 기본 기준은 worst case다.

> 정답

개념은 힌트 참고.

if가 일부 반복에서만 실행돼도, **worst case 기준**으로는 매 반복 실행되는 입력을 기준으로 분석한다 (실현 가능한 최악의 실행 경로의 비용).

정답: Worst-case 분석에서는 크기 n인 입력 중 실제로 가능한 최대 실행 비용을 구한다. if문의 본문이 매 반복 실행될 수 있다면 그 경우를 사용할 수 있지만, 조건의 구조상 불가능하면 실제로 실행 가능한 최대 횟수를 계산해야 한다. 따라서 조건문을 매번 실행한다고 가정하는 방법은 상한을 구하는 데 사용할 수 있으나, 항상 타이트한 worst-case 복잡도를 주지는 않는다.

7. Amortized(분할 상환) Analysis (분할 상환)

동기

- `put(x)` 연산이 가끔 배열 확장 필요 → worst case $O(n)$이지만 매번 그런 건 아님
- **평균 비용**을 n번 call에 대해 계산:

$$\text{amortized cost} = \frac{\sum_{i=1}^{n}(\text{cost of i-th call})}{n}$$

두 버전 비교

Version 1: 크기 +5씩 증가 (상수 증가)

확장은 5번마다 → 5, 10, 15, ... 크기 복사. **총 비용 유도** ($n = 5m$ 번 put):

```
Version 1: 배열 크기를 5씩 증가
총 n = 5m번의 put 연산 수행
put 자체 비용: 매번 1 → 총 n
배열 확장 시 기존 원소 복사 비용:
복사량 = 0+5+10+...+5(m-1)
복사량 = 5·m(m-1)/2
정확한 총비용:
T(n) = n + 5·m(m-1)/2
= n + (5/2)·(n/5)·(n/5 - 1)
= n + n²/10 - n/2
= n²/10 + n/2
따라서 총비용: T(n) = Θ(n²)
put 1회당 평균 비용:
T(n)/n = (n²/10 + n/2)/n
= n/10 + 1/2
= Θ(n)
결론: put의 분할상환 비용(amortized cost)은 Θ(n)
배열 크기를 5씩 증가시키면 put 1회당 평균 비용도 n에 비례하여 증가하므로, 상수 시간 O(1)을 보장하지 못함.
```

Version 2: 크기 × 2 (더블링)

Version 2 — 크기를 2배로 증가: 초기 용량을 1로 두고 배열이 가득 찼을 때 용량을 두 배로 늘린다. 확장은 2, 3, 5, 9, ⋯ 번째 put에서 발생하며, 각 확장에서 복사하는 원소 수는 1, 2, 4, 8, ⋯이다. $n=2^k$번의 put에 필요한 복사 비용의 합은 $1+2+\cdots+2^{k-1} = n-1$이다. 기본 삽입 비용을 더하면 총비용은 $T(n)=n+(n-1)=2n-1$이므로 호출당 분할 상환 비용은 $T(n)/n = 2-1/n = \Theta(1)$이다. 임의의 n에 대해서도 복사 비용의 합이 O(n)으로 제한되므로 호출당 분할 상환 비용은 Θ(1)이다.

```
Total ≈ n + (1 + 2 + 4 + ... + 2^(k-1))
    ≈ n + (2^k - 1)    ← 기하급수 합
    ≈ 2n - 1

amortized = (2n-1)/n = 2 - 1/n = O(1)
```

→ **call당 O(1)**. 좋음.

교훈

> **상수 증가 → $O(n)$ per call, 더블링 → $O(1)$ per call**. 같은 worst case $O(n)$이라도 amortized 관점에서 전혀 다른 성능.

Summary

개념	핵심
Algorithm(알고리즘)	finite-time step 절차
Empirical(경험적 분석) vs Theoretical(이론적 분석)	실측 vs 수학 분석
Best(최선)/Avg/Worst(최악)	Best/Avg/Worst = 입력에 따른 최선·평균·최악 비용. O/Ω/Θ = 점근적 상한·하한·타이트 경계. 두 분류는 독립적이다.
Big-O(빅오)	상한 (upper bound)
Amortized(분할 상환)	여러 call의 평균 (doubling 효과)